



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2025), B.Sc. in CSE (Day)*

Lab Report No #02
Course Title: Machine Learning Lab
Course Code: CSE-412
Section: 222_D1

Lab Experiment Name: K-Means Clustering and Supervised Classification

Students Details

Name	ID
Md.Zehadul Islam	222902069

Submission Date: July 21, 2025
Course Teacher's Name: Md. Abu Rumman Refat

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Lab Report-II	2
1.1	TITLE OF THE LAB REPORT EXPERIMENT	2
1.2	OBJECTIVES	2
1.3	PROCEDURE	2
1.4	IMPLEMENTATION	3
1.4.1	Step 1: Data Processing	3
1.5	DISCUSSION	7

Chapter 1

Lab Report-II

1.1 TITLE OF THE LAB REPORT EXPERIMENT

K-Means Clustering and Supervised Classification

1.2 OBJECTIVES

1. To apply K-Means clustering on a labeled dataset (Iris) and measure clustering accuracy.
2. To determine the optimal number of clusters (K) by averaging accuracy over multiple runs.
3. To evaluate the effect of different train-test split ratios on classification accuracy using a Decision Tree classifier.

1.3 PROCEDURE

Dataset Preparation

- Loaded the Iris dataset and converted class labels to numeric using `LabelEncoder`.
- Normalized features using `StandardScaler`.

K-Means Clustering

- Applied K-Means with $K = 3$ clusters.
- Predicted cluster labels and mapped them to actual labels.
- Calculated clustering accuracy.

Finding Best K

- Tested K values from 2 to 10.
- For each K, ran clustering 10 times and recorded accuracy.
- Computed average accuracy to find the optimal K.

Train-Test Split Experiment

- Used DecisionTreeClassifier for classification.
- Tested different train-test ratios (e.g., 90:10, 80:20, ..., 10:90).
- Calculated and plotted accuracy for each split.

Visualization

- Plotted K vs Average Accuracy and Test Size vs Accuracy for analysis.

1.4 IMPLEMENTATION

1.4.1 Step 1: Data Processing

Listing 1.1: The dataset was split into features and labels zero mean and unit variance

```

1 X = df.drop('species', axis=1)
2 y = df['species']
3
4 from sklearn.preprocessing import StandardScaler
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(X)

```

Listing 1.2: KMeans Clustering Accuracy Calculation

```

1 k = len(np.unique(y))
2 kmeans = KMeans(n_clusters=k, random_state=42)
3 clusters = kmeans.fit_predict(X_scaled)
4
5 labels = np.zeros_like(clusters)
6 for i in range(k):
7     mask = (clusters == i)
8     labels[mask] = mode(y[mask])[0]
9
10 acc_kmeans = accuracy_score(y, labels)
11 print(f"          Part 1 - KMeans Accuracy (K={k}): {acc_kmeans
    :.4f}")

```

 ♦ Part 1 - KMeans Accuracy (K=3): 0.6667

Figure 1.1: Output of Print Features and Labels in DataFrame

Listing 1.3: Determining the Best K

```
1 k_range = range(2, 11)
2 accuracy_k = []
3
4 for k in k_range:
5     acc_list = []
6     for seed in range(10):
7         km = KMeans(n_clusters=k, random_state=seed)
8         cl = km.fit_predict(X_scaled)
9
10        lbls = np.zeros_like(cl)
11        for i in range(k):
12            mask = (cl == i)
13            if np.sum(mask) > 0:
14                lbls[mask] = mode(y[mask])[0]
15
16        acc_list.append(accuracy_score(y, lbls))
17
18    avg_acc = np.mean(acc_list)
19    accuracy_k.append(avg_acc)
20    print(f"K={k}      Avg Accuracy: {avg_acc:.4f}")
```

 K=2 → Avg Accuracy: 0.6667
K=3 → Avg Accuracy: 0.8213
K=4 → Avg Accuracy: 0.8360
K=5 → Avg Accuracy: 0.8500
K=6 → Avg Accuracy: 0.8620
K=7 → Avg Accuracy: 0.8813
K=8 → Avg Accuracy: 0.8733
K=9 → Avg Accuracy: 0.9033
K=10 → Avg Accuracy: 0.9320

Figure 1.2: Best K Determination from 10 Runs

Listing 1.4: A plot was generated to visualize the performance:

```
1 import matplotlib.pyplot as plt
2
3 plt.plot(k_range, accuracy_k, marker='o')
4 plt.title("Best K via 10 Run Accuracy")
5 plt.xlabel("Number of Clusters (K)")
6 plt.ylabel("Average Accuracy")
7 plt.grid(True)
8 plt.show()
```

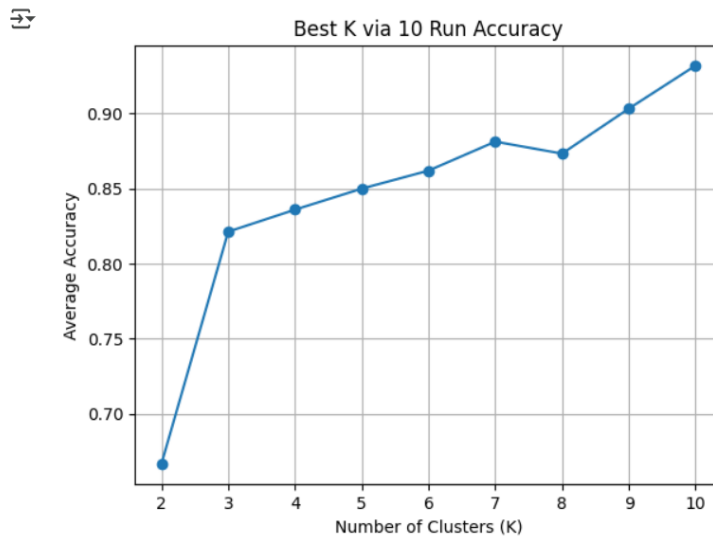


Figure 1.3: Output of Average Accuracy vs K Plot.

Listing 1.5: A Best Train-Test Split Ratio for Classification

```
1 ratios = [0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9]
2 acc_ratio = []
3
4 for r in ratios:
5     X_train, X_test, y_train, y_test = train_test_split(
6         X_scaled, y, test_size=r, random_state=42)
7     model = DecisionTreeClassifier()
8     model.fit(X_train, y_train)
9     y_pred = model.predict(X_test)
10    acc = accuracy_score(y_test, y_pred)
11    acc_ratio.append(acc)
12    print(f"Test Ratio={r}      Accuracy: {acc:.4f}")
```

```

⇒ Test Ratio=0.1 → Accuracy: 1.0000
   Test Ratio=0.2 → Accuracy: 1.0000
   Test Ratio=0.3 → Accuracy: 0.9556
   Test Ratio=0.4 → Accuracy: 0.9667
   Test Ratio=0.5 → Accuracy: 0.9333
   Test Ratio=0.6 → Accuracy: 0.9667
   Test Ratio=0.7 → Accuracy: 0.9238
   Test Ratio=0.8 → Accuracy: 0.9333
   Test Ratio=0.9 → Accuracy: 0.9407

```

Figure 1.4: Best Train-Test Split Ratio

Listing 1.6: A plot was created to visualize how the test size affected model accuracy:

```

1 plt.plot(ratios, acc_ratio, marker='s', color='green')
2 plt.title("Test Ratio vs Accuracy (Decision Tree)")
3 plt.xlabel("Test Size Ratio")
4 plt.ylabel("Accuracy")
5 plt.grid(True)
6 plt.show()

```

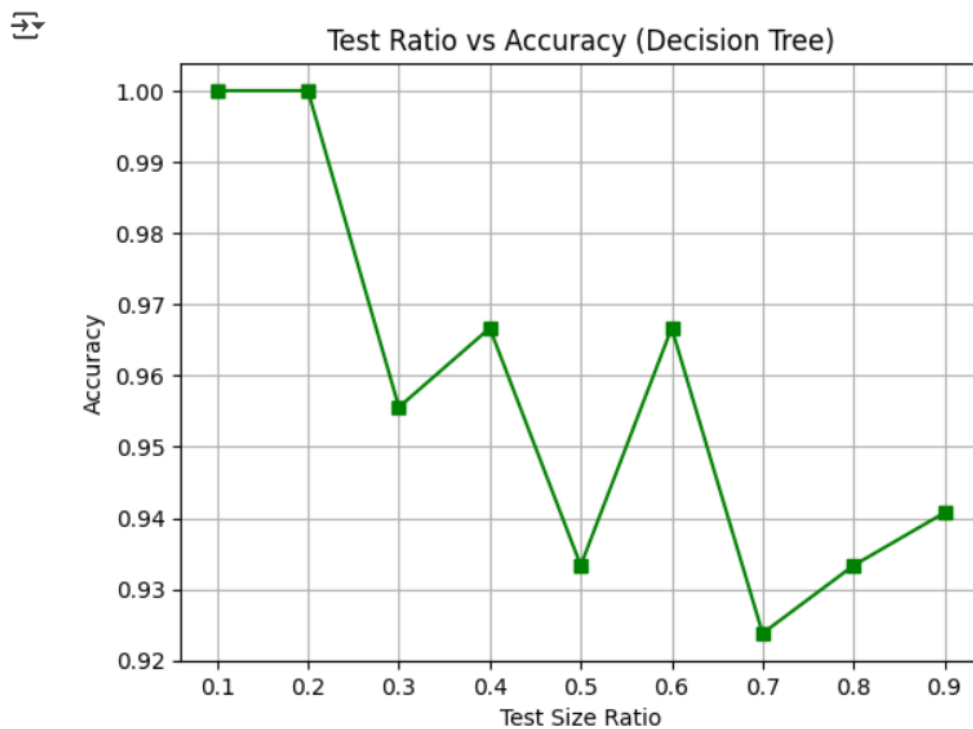


Figure 1.5: Output of Accuracy vs Test Ratio Plot.

1.5 DISCUSSION

This lab taught me valuable insights into both unsupervised and supervised learning. We used K-Means to cluster iris flowers and got good accuracy even without labels. After testing different K values, $K = 3$ gave the best results, which matches the real number of classes. Then, using a Decision Tree, we tried different train-test splits. Around 70–80% training gave better accuracy. Overall, both clustering and classification worked well on this dataset.